

A Project Report on

**Client-Side Detection of Indirect Prompt Injection and
Unauthorized Data Exfiltration in Browser-to-AI Interactions
Using Multi-Indicator Content Analysis**

Submitted in partial fulfilment for award of degree of

Master of Technology

In Cybersecurity

Submitted by

Sushanth Sridhar

SRN: R24TF007 | Batch: CS14

Under the Guidance of

Sandeep

[Designation]

REVA Academy for Corporate Excellence

REVA University

Rukmini Knowledge Park, Kattigenahalli,

Yelahanka, Bangalore – 560064

July 2026

Candidate's Declaration

I, Sushanth Sridhar, hereby declare that I have completed the project work towards Master of Technology in Cybersecurity at REVA University on the topic entitled “Client-Side Detection of Indirect Prompt Injection and Unauthorized Data Exfiltration in Browser-to-AI Interactions Using Multi-Indicator Content Analysis” under the supervision of Sandeep, [Designation]. This report embodies the original work done by me in partial fulfilment of the requirements for the award of the degree for the academic year [Year].

Place: Bengaluru

Name of the Student: Sushanth Sridhar

Date:

Signature of Student

Acknowledgment of Project Ownership and Usage Rights

I, Sushanth Sridhar, a student enrolled in the Master of Technology in Cybersecurity Program and Batch CS14 at RACE, hereby acknowledge that any project, including but not limited to software, hardware, research, or other intellectual property created by me during my academic tenure at RACE, is the property of RACE, REVA University.

I understand and agree that RACE has the exclusive rights to use, reproduce, modify, or distribute the aforementioned projects for academic, research, and further development purposes. This includes the right to monetize, commercialize, or otherwise exploit the projects as deemed fit by RACE.

I acknowledge that I have no objection to RACE, REVA University, using, reproducing, or further developing the projects for the benefit of the institution and its academic community. I further affirm that any commercial or research activities related to the projects conducted by RACE shall not require additional consent or approval from me.

This acknowledgment is made willingly and without any reservations. I am grateful for the education and opportunities provided by RACE, and I recognize the importance of contributing to the academic and research goals of the institution.

Place: Bengaluru

Name of the Student: Sushanth Sridhar

Date:

Signature of Student

Certificate

This is to certify that the project work entitled “Client-Side Detection of Indirect Prompt Injection and Unauthorized Data Exfiltration in Browser-to-AI Interactions Using Multi-Indicator Content Analysis” has been carried out by Sushanth Sridhar with SRN R24TF007, who is a bonafide student of REVA University, submitting the second-year project report in fulfilment for the award of Master of Technology in Cybersecurity during the academic year [Year]. The project report has been tested for plagiarism and has passed the plagiarism test with a similarity score of less than 15%. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Signature of the Guide

Signature of the Director

Sandeep

[Name of the Director]

Guide

Director

External Viva

Names of the Examiners

[Name] [Designation] [Signature]

[Name] [Designation] [Signature]

Place: Bengaluru

Date:

Acknowledgment

I would like to express my sincere gratitude to my project guide, Sandeep, for the continuous guidance, technical feedback, and encouragement provided throughout this capstone project.

I am also thankful to the faculty and program office of REVA Academy for Corporate Excellence (RACE) for their support during the course of this MTech in Cybersecurity program, batch CS14.

I extend my gratitude to my classmates and peers for their valuable discussions and feedback during the development and evaluation of this project. I am also thankful to my family and friends for their patience and encouragement throughout this journey.

I gratefully acknowledge the support provided by the Hon'ble Chancellor, Dr. P Shayma Raju, Hon'ble Vice Chancellor, Dr. Sanjay Chitnis, and Registrar, Dr. M. Dhanamjaya, REVA University, whose vision for research-driven, industry-relevant postgraduate education made this capstone project possible.

Place: Bengaluru

Date:

Similarity Index Report

This is to certify that this project report titled “Client-Side Detection of Indirect Prompt Injection and Unauthorized Data Exfiltration in Browser-to-AI Interactions Using Multi-Indicator Content Analysis” was scanned for similarity detection. The process and outcome are given below. The plagiarism report is attached in the appendix.

- Software Used: Turnitin

Date of Report Generation: [to be filled after running Turnitin]

Similarity Index in %: [to be filled — must be < 15%]

Total word count: [auto-filled by Word once final]

Name of the Guide: Sandeep

Place: Bengaluru

Name of the Student: Sushanth Sridhar

Date:

Signature of Student

Verified by:

Signature

[Verifying officer name and title to be added]

List of Abbreviations

Abbreviation	Expansion
DOM	Document Object Model
TLS	Transport Layer Security
SNI	Server Name Indication
JA3 / JA4	TLS client fingerprinting methods (Salesforce JA3; FoxIO JA4)
MV3	Chrome Extension Manifest Version 3
CDP	Chrome DevTools Protocol
DLP	Data Loss Prevention
PII	Personally Identifiable Information
EDR	Endpoint Detection and Response
MITRE ATLAS	Adversarial Threat Landscape for Artificial- Intelligence Systems
OWASP	Open Worldwide Application Security Project
F1	Harmonic mean of precision and recall
TP / FP / TN / FN	True Positive / False Positive / True Negative / False Negative

List of Figures

[Populated at final assembly, Stage 6 — see Fig labels generated throughout the report]

List of Tables

[Populated at final assembly, Stage 6 — see Table labels generated throughout the report]

Abstract

(Not to exceed 1 page)

AI-powered browser agents (ChatGPT Atlas, Perplexity Comet, Claude in Chrome) now read and act on arbitrary web content on a user's behalf, and the open web has begun filling with content specifically crafted to hijack that behaviour — indirect prompt injection hidden in off-screen CSS, HTML comments, and metadata that a human never sees but an agent's context window does. Independent testing found ChatGPT Atlas blocked only 5.8% and Perplexity Comet only 7% of such pages, against 47–53% for conventional phishing defences in ordinary browsers, and Google recorded a 32% rise in malicious indirect-injection content between November 2025 and February 2026. Existing defences are vendor-side and reactive; a proactive, client-side sentinel that inspects page content before an agent ever reads it is largely unexplored.

This project builds and evaluates such a sentinel as a standalone system: a Chrome extension that scans the live DOM of any page for six independent indicators of hidden instructions (off-screen positioning, zero-width Unicode, suspicious HTML comments, hidden alt/ARIA text, JSON-LD metadata, and visible imperative language) and combines them with a noisy-OR multi-indicator score; a Go agent bridging the extension to a local Python scoring service and a Postgres store via Chrome's native messaging API; a real Zeek/Suricata network sensor performing JA3/JA4 TLS fingerprinting to identify known AI platforms and cluster unlisted “shadow AI” services; and an outbound data-loss-prevention gate that intercepts and classifies fetch/XHR bodies to known AI domains before they are sent, holding flagged content for user approval.

The system was evaluated end to end — not simulated — against a 70-page labelled synthetic dataset (30 benign, 10 deliberately weak “hard-negative” pages, 30 injected) visited by a 4-container test fleet, each container its own OS user and hostname, running the real extension, agent, and scoring pipeline. The multi-indicator detector (C) achieved F1 0.979 (precision 0.983, recall 0.975), against F1 0.911 for a visibility-only baseline (B) and F1 0.784 for a keyword-only baseline (A) — mirroring the same $C > B > A$ pattern from the author's prior capstone on network-based C2 detection. The result that most directly supports the multi-indicator design: on the hard-negative pages, the keyword-only baseline false-positived on 32 of 40 visits and the visibility-only baseline on 20 of 40, against 2 of 40 for the multi-indicator detector.

Building the system end to end surfaced nine real, documented defects and their fixes spanning every layer — a Chrome install-path assumption that silently broke content-script injection, a native-messaging registration gap, a TLS client-fingerprint (JA3) instability caused by Chrome's GREASE mechanism that silently defeated the shadow-AI clustering rule until re-keyed to JA4, and others — each caught by checking real output against expectation rather than trusting a clean-looking log, and each recorded with root cause and fix rather than smoothed over in the final narrative. The shadow-AI clustering rule was also confirmed to have genuine practical limits: once corrected, the same fingerprint that caught the two seeded “unknown AI” test domains also pulled in sixteen unrelated domains from the browser's own background traffic, demonstrating directly why the technique is a human triage signal rather than an automated verdict.

The findings support the conclusion that combining independent, cheaply-computed content indicators client-side detects indirect prompt injection substantially more reliably than any single indicator alone, while remaining fully inspectable — every flagged page shows exactly which indicators fired — and that this client-side vantage point is structurally unavailable to network-only or vendor-side defences.

Keywords: Prompt Injection, Indirect Prompt Injection, Browser AI Agents, Data Loss Prevention, Shadow AI, TLS Fingerprinting, JA3, JA4, MITRE ATLAS, Multi-Indicator Detection, Chrome Extension Security

Table of Contents

Right-click here and choose "Update Field" (or press F9) to generate the Table of Contents with page numbers.

Chapter 1: Introduction

1.1 The Shift From a Network Perimeter to an Agentic Browser Perimeter

Most existing web security controls assume the browser is a passive rendering engine — a human reads a page and decides what to do next. Through late 2025 and into 2026 that assumption broke down: a new class of agentic AI browser (OpenAI's ChatGPT Atlas, Perplexity's Comet, Anthropic's Claude in Chrome) reads pages and takes actions — clicking, filling forms, sending data — on the user's behalf, with no human necessarily reading the page at all.

Attackers adapted immediately. Content is now deliberately crafted to be read by an AI agent rather than a human — instructions hidden in off-screen CSS, HTML comments, zero-width Unicode, and page metadata, invisible on screen but present in the DOM an agent's context window ingests. This technique is called indirect prompt injection, and it is not theoretical: Google recorded a 32% rise in malicious indirect-injection content between November 2025 and February 2026 [2]; independent red-team testing found ChatGPT Atlas blocked only 5.8% and Perplexity Comet only 7% of malicious pages designed to hijack an agent, against 47% and 53% for conventional phishing defences in ordinary Chrome and Edge [6]; and OpenAI's own security leadership has stated that prompt injection may never be fully solved for browser agents [7].

This is structurally the same detection problem the author's prior capstone addressed for network-based command-and-control (C2) traffic — an attacker hiding instructions inside a channel the defender already trusts and cannot simply block — relocated to a new layer. There, the trusted channel was DNS/HTTPS traffic and the hidden payload was a beacon to a C2 server; here, the trusted channel is ordinary web content and the hidden payload is an instruction targeting an AI agent's context window instead of a network socket.

The same agentic-AI adoption wave creates two adjacent, related problems inside the exact same browser-to-AI channel. First, organisations frequently do not know which AI services their own browsers are talking to — an increasingly recognised class of shadow IT called “shadow AI”. Second, employees paste sensitive data into AI chat interfaces with no governance at all: industry reporting places the browser as the point of roughly 80% of generative-AI data leaks, with employees regularly pasting customer data, credentials, and PII directly into prompts [11]. Commercial data-loss-prevention (DLP) products for exactly

this problem already exist (Strac, Nightfall, Microsoft Purview) [11], but none of the literature or tooling surveyed in Chapter 2 combines platform discovery, outbound DLP, and proactive client-side injection detection in a single evaluated system.

1.2 Scope and Organization of This Report

This report documents the design, implementation, and evaluation of Browser AI Sentinel, a standalone Chrome extension and local agent system addressing all three problems above. Chapter 2 reviews the relevant literature and establishes the gap this project addresses. Chapter 3 states the problem precisely, including what is explicitly out of scope. Chapter 4 lists the study's objectives. Chapter 5 describes the phased build-and-verify methodology actually used, the A/B/C evaluation methodology, and — in the interest of the same evidence-over-narrative standard applied throughout this project — the real problems encountered while building it and how each was found and fixed. Chapter 6 lists resource requirements. Chapters 7 and 8 cover software design and implementation. Chapter 9 covers testing and validation, and Chapter 10 presents the analysis and results, including the real precision/recall/F1 numbers from a live, non-simulated 4-endpoint test fleet. Chapter 11 closes with limitations and future scope.

Chapter 2: Literature Review

2.1 Indirect Prompt Injection Against AI Browser Agents

Indirect prompt injection occurs when a system composes untrusted external content (a web page, a document, an email) with trusted instructions in a single context window and lets a language model act on the result [1]. Brave's security research characterises this as a fundamental, currently unsolved challenge for any AI system that ingests third-party content [1]. Field observations from Zscaler ThreatLabz [5] and Palo Alto Networks' Unit 42 [4] document real campaigns hiding instructions using off-screen CSS positioning, HTML comments, and structured JSON-LD metadata that machines parse as trusted context but a human reader never sees — the exact technique categories this project's DOM scanner targets. Mozilla has separately warned of the same risk class specifically for AI coding agents [3], and Google's own telemetry showed a 32% increase in malicious indirect-injection content in a four-month window spanning late 2025 and early 2026 [2].

Independent, adversarial testing of production AI browsers found ChatGPT Atlas and Perplexity Comet blocking only 5.8% and 7% of malicious pages respectively, against 47–53% for conventional browsers defending against phishing [6]. OpenAI's own head of preparedness has stated publicly that prompt injection for browser agents “may never be fully solved” [7], and OpenAI has since shipped iterative hardening for Atlas specifically in response to red-team findings [8]. Taken together, this literature establishes indirect prompt injection as a live, growing, and — by the vendors' own admission — structurally difficult problem, not a hypothetical one.

2.2 Existing Detection and Red-Team Tooling

Existing defensive and offensive tooling for prompt injection is concentrated at two other layers, neither of which is the client-side DOM. Open-source detectors such as Rebuff [17] and Pytector operate on the prompt text itself, using heuristics, a fine-tuned classifier, or an LLM-as-judge to flag a string as adversarial before it reaches a model — useful for direct injection into an application's own input field, but not applicable to instructions hidden in a third-party web page's DOM. Offensive/red-team tools (Praetorian's Augustus [16], a Burp Suite extension for LLM injection testing, and in-browser LLM-guided fuzzing research) are built to discover injection vulnerabilities in a target system, not to protect an end-user's own browsing session. Academic benchmarks — WAIInjectBench [9], PromptShield, and a 2026

study showing prompt-injection detection performance is strongly regime-dependent (a detector tuned for one deployment context does not transfer cleanly to another) [10] — evaluate detectors against curated text/image corpora, not against a live DOM as an AI agent would actually encounter it. No surveyed system runs proactively, client-side, on the rendered page itself, ahead of an agent reading it — the specific gap this project addresses.

2.3 Shadow AI Discovery and Enterprise DLP for AI Tools

“Shadow AI” — employees using AI services an organisation has not sanctioned or even identified — is now a recognised commercial security category. Vendors including Strac, Nightfall, and Microsoft Purview ship browser-extension-based DLP that scans outbound prompts and file uploads to known AI domains before they are sent, and report that the browser is where the large majority of generative-AI data leakage actually occurs [11]. These products are mature for the outbound-DLP half of the problem — this project's own DLP module (Chapter 8) is explicitly positioned as governance, not a novel research contribution, for that reason. Discovery of shadow AI itself, however, is uniformly domain-list-based in the surveyed products: an AI service is “known” only if a vendor has already catalogued its domain. None of the surveyed tools attempt network-layer fingerprint-based discovery of *unlisted* AI services, which is where this project's shadow-AI clustering module (Chapters 8 and 10) differs — and, as Chapter 10 shows honestly, where a fingerprint-based approach's real practical precision limits also become visible.

2.4 TLS Client Fingerprinting (JA3/JA4)

JA3, developed at Salesforce [14], fingerprints a TLS client by hashing the cipher suites and extensions offered in its ClientHello message. JA4, from FoxIO [15], is a newer, explicitly GREASE-aware successor designed to strip out the randomised reserved values modern browsers insert specifically to prevent fingerprint ossification. The author's prior capstone used JA3/JA4 fingerprinting as one of eight behavioural indicators for detecting C2 beaconing over encrypted traffic. This project repurposes the same underlying technique for a different task — AI-platform identification and shadow-AI discovery instead of anomaly-based C2 detection — and, in the course of doing so (Chapter 5.3), independently rediscovered in a real, verified fleet-testing scenario exactly why JA4's GREASE-awareness matters: JA3 alone proved unreliable as a stable client identifier for real Chrome traffic.

2.5 MITRE ATLAS and OWASP LLM Top 10

MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) [12] is a living, ATT&CK-modelled knowledge base of adversary tactics and techniques specifically against AI-enabled systems, covering 16 tactics and over 80 techniques as of its most recent revision. This project's injection-detection module maps cleanly to ATLAS technique AML.T0051.001 (LLM Prompt Injection: Indirect, under the Initial Access tactic) — verified against multiple independent sources during threat-model scoping. The outbound-DLP module's mapping to ATLAS is, honestly, unresolved: secondary sources disagreed on what technique AML.T0025 actually names, and the authoritative atlas.mitre.org site could not be fetched directly to confirm (a JavaScript single-page application). OWASP's Gen AI Security Project LLM Top 10, specifically LLM02:2025 “Sensitive Information Disclosure” [13], is used as the confirmed, safer interim citation for that module instead of an unverified ATLAS ID — a deliberate choice to avoid citing something not actually checked.

2.6 Consolidated Gap and This Project's Contribution

Across the literature surveyed, three observations converge. First, indirect prompt injection against AI browser agents is a real, growing, and — by vendors' own admission — structurally unsolved problem. Second, existing defensive tooling operates either on raw prompt text or as offensive red-team tooling, not as a proactive client-side DOM sentinel running ahead of an agent reading a page. Third, shadow-AI discovery in commercial DLP products is domain-list-based, with no surveyed system attempting network-fingerprint-based discovery of unlisted AI services. This project's contribution is a single, standalone system addressing all three gaps together — proactive DOM-level multi-indicator injection detection, JA4-based shadow-AI discovery, and an outbound DLP gate — built and evaluated end to end against real, non-simulated traffic through an actual multi-endpoint test fleet, with every claim in this report backed by a stored number, a screenshot, or a citation rather than prose alone.

Chapter 3: Problem Statement

AI browser agents ingest the content of arbitrary web pages into their working context and act on it, but the current generation of such agents demonstrably fails to distinguish content intended for a human reader from content specifically crafted to instruct the agent itself. No control exists at the point where this matters most — the browser, before the agent's own (often opaque, vendor-controlled) reasoning pipeline processes the page — to flag content that reads like an instruction targeting an AI rather than a human. Separately, the same browser-to-AI channel is used to send arbitrary, ungoverned data — including personally identifiable information and secrets — to third-party AI services with no visibility or approval step, and organisations frequently cannot enumerate which AI services their own users' browsers are actually talking to.

This project's threat model is scoped precisely, and deliberately excludes adjacent problems that would each need a different architecture to address correctly:

- In scope: indirect prompt injection — malicious instructions hidden in page content (off-screen CSS, zero-width Unicode, HTML comments, hidden alt/ARIA text, JSON-LD metadata) that a human browsing normally would not see but an AI agent reading the DOM would ingest.
- In scope: identification of which AI platform a browser is communicating with, including platforms not on any predefined list (“shadow AI”), via network-layer TLS fingerprinting rather than a static domain allowlist alone.
- In scope: unauthorized transmission of sensitive data (PII, credentials, secrets) from the browser to a known AI service, intercepted and gated before the request leaves the machine.
- Out of scope: injection via the user's own typed prompts (a different, already well-covered problem — application-level prompt filtering — and not the novel contribution here).
- Out of scope: post-hoc analysis of an AI agent's response after a successful compromise — this would require access to the agent's internal state, which is not available to a client-side browser extension.
- Out of scope: blocking an AI agent that reads a page through its own privileged channel (e.g. the Chrome DevTools Protocol or an accessibility-tree API) rather than the visible DOM — this system is explicitly an intrusion detection system (IDS), not a prevention

system (IPS), for that reason, exactly mirroring the posture the author's prior capstone took with Suricata/Zeek at the network layer.

Chapter 4: Objectives of the Study

The study has four objectives, each mapped directly to an implemented module (Chapter 8) and a verification step (Chapter 9):

- Objective 1: Design and implement a client-side, multi-indicator DOM scanner that detects indirect prompt injection with materially better precision and recall than any single indicator used in isolation, and evaluate this claim against a real labelled dataset rather than assert it.
- Objective 2: Design and implement network-layer AI-platform identification via real TLS SNI/JA3/JA4 fingerprinting, including a heuristic for discovering AI services not on any predefined list (“shadow AI”), and evaluate the heuristic's real behaviour against live traffic rather than only synthetic test cases.
- Objective 3: Design and implement an outbound data-loss-prevention gate that intercepts and classifies content bound for known AI services before it is sent, holding PII/secret-bearing requests for explicit user approval.
- Objective 4: Build a small, realistic multi-endpoint test environment (distinct simulated users/hosts) that exercises all three modules through the actual running system — not a simulation — and produce a governance-style dashboard summarising the results, modelled on real EDR/CASB product patterns (CrowdStrike Falcon, SentinelOne Singularity, Microsoft Purview) researched during design.

Chapter 5: Project Methodology

5.1 Phased Build-and-Verify Methodology

The system was built in four sequential phases, each fully verified against real running software before the next began — the same build-verify-before-proceeding discipline used in the author's prior capstone, applied here to a very different architecture.

- Phase 1 — single-endpoint MVP: the Chrome extension, Go agent (daemon + native-messaging shim), Python scoring service, and Postgres store, wired together and verified end to end in a real (not simulated) Chrome instance via the Chrome DevTools Protocol, with the AI-platform module still a static domain-list stub.
- Phase 2 — real network sensor: a standalone Zeek/Suricata deployment (fresh install and configuration, no code or infrastructure shared with the author's prior capstone) replacing the Phase 1 stub with real SNI/JA3/JA4 extraction and a first-cut shadow-AI clustering heuristic.
- Phase 3 — labelled dataset and multi-endpoint test fleet: a 70-page synthetic, labelled dataset for the injection-detection module, and four Docker containers — each its own OS user and hostname — running the real extension, agent, and scoring pipeline against real (not mocked) traffic, producing the evaluation data in Chapter 10.
- Phase 4 — dashboard: a governance-style web dashboard over the accumulated real data, backed by new server-side aggregate API endpoints rather than raw data dumped to the client.

5.2 Evaluation (A/B/C Comparison) Methodology

The injection-detection module is evaluated using the same three-configuration comparison methodology as the author's prior capstone, applied to a different indicator set: Configuration A is a keyword-only baseline (flags a page if any visible text matches an imperative-to-AI phrase pattern); Configuration B is a visibility-only baseline (flags a page if any off-screen or zero-width-Unicode indicator is present, regardless of content); Configuration C is the full multi-indicator detector, which combines all six indicators using a noisy-OR combination (Chapter 8.1) rather than a simple threshold sum. All three configurations are computed from the same underlying indicator counts in a single request to the scoring service, so A, B, and C are never subject to timing or environment differences relative to one another — only to the real page content itself.

Ground truth for the evaluation is a synthetic, labelled dataset (70 pages: 30 benign, 10 “hard-negative” pages carrying exactly one weak indicator, 30 injected pages carrying two to four indicators), generated deterministically so the same dataset can be regenerated and re-scored. Precision, recall, and F1 are computed per configuration from real confusion-matrix counts recorded by the live system during a real 4-endpoint fleet run (Chapter 9), not simulated or hand-computed.

5.3 Problems Encountered and How They Were Resolved

In keeping with the same standard applied throughout this project — real evidence, not a polished-sounding narrative — this section records every material defect found while building the system, its root cause, and its fix, in the order each was discovered. Several were only found by checking real output against expectation rather than trusting a clean-looking log or a successful build.

Chrome install path. `--load-extension` (command-line flag) does not reliably install an unpacked Manifest V3 extension on the Chrome version used in this project — a service worker spins up, but content scripts never inject. Root cause: this flag is effectively degraded on recent Chrome; the correct path is the `Extensions.loadUnpacked` DevTools Protocol command (the same one `chrome://extensions`'s “Load unpacked” button uses internally), which also correctly honours the manifest's pinned signing key for a deterministic extension ID. Fixed by switching every automated install path to that command.

Native messaging host discovery. A Chrome profile launched with a custom `--user-data-dir` does not discover a native-messaging-host registration placed only at the user-level path; only the system-wide path was reliably found. Fixed by registering the host at both locations.

Zeek silently drops all TLS. The virtio network interface on the deployment VM has checksum offloading enabled, which makes Zeek see “invalid” TCP checksums on outbound packets and discard them by default — confirmed by comparing `conn.log` (populated) against `ssl.log` (empty) on the same capture. Fixed with Zeek's `-C` (ignore-checksums) flag.

Shadow-AI confidence default. The `shadow_ai_clusters.confidence` column's schema default was mistakenly `'candidate'` instead of `'observed'`, so every single-domain sighting looked like a multi-domain cluster immediately — caught by testing against

real ambient background traffic on the deployment machine, not only the constructed happy path. Fixed in the schema and via a one-time correction on the live database.

Suricata multi-interface capture. Passing `-i ens18` on the command line silently overrides a YAML config's multi-interface `af-packet` list down to a single interface. Fixed by using `--af-packet` with no value, which tells Suricata to read the interface list from the config file.

Same-host traffic invisible to the sensor. The mock “unknown AI” test endpoints run on the same host as the sensor; same-host traffic to a local address never transits the physical network interface, so it was invisible to a sensor watching only `ens18`. Fixed by adding a second Zeek instance and a second Suricata interface, both watching loopback.

Container native-messaging gap repeated. The same native-messaging registration gap found earlier on the host was independently reintroduced inside the Phase 3 test-fleet containers, because the fix was not carried over into the container entrypoint script — causing the first full fleet run to silently record zero rows despite every page reporting a successful visit. Fixed the same way, plus each container's native-messaging shim needed to be told which port its own daemon was listening on, since containers share ports 1-to-1 with the host under the chosen networking mode.

Extension silently skipped benign pages. The DOM scanner only messaged the local agent when it found at least one indicator, so the dataset's 30 pure-benign pages were never scored or logged at all — leaving no true-negative data for the evaluation even after the storage layer was changed to log every score. Fixed by removing the early return so every scan reports, clean or not.

Hard-negative dataset miscalibration. The zero-width-Unicode hard-negative test pages were generated with four zero-width characters, which the scorer's own weighting treats as a maximally strong single indicator — not the intended weak, single-occurrence test case — causing that indicator alone to saturate the score. Fixed by parameterising the generator so hard-negative pages use exactly one occurrence.

JA3 instability under Chrome's GREASE. Shadow-AI clustering, keyed on the pair (JA3, JA4), never accumulated multi-domain evidence for real browser traffic. Root cause: Chrome's GREASE mechanism randomises reserved cipher/extension values in every ClientHello, which JA3's hashing treats as signal — confirmed empirically, with ten of

twelve real Chrome connections to the same two test domains each producing a distinct JA3, while JA4 (designed to strip GREASE before hashing) stayed identical across all of them. Earlier testing had only appeared to work because it used `curl`, which does not implement GREASE. Fixed by re-keying the clustering table on JA4 alone.

Postgres ORDER BY on combined aliases. An `ORDER BY` expression combining two `SELECT`-list aliases from correlated subqueries failed with “column does not exist,” even though each alias resolves individually. Fixed by wrapping the query in a subquery and ordering the outer query instead.

Host-level, not per-container, sensor attribution. Network-sensor-derived events are attributed to whichever endpoint's agent is tailing the sensor logs — only the host, since the test-fleet containers do not run their own sensor — so a shadow-AI event generated by one container's browsing is recorded against the host, not that container. This is a genuine architectural limitation of packet-capture-based attribution, not a bug to be silently fixed; it is stated plainly in Chapter 10 and Chapter 11 rather than left for a reviewer to discover.

Chapter 6: Resource Requirement Specification

6.1 Hardware Requirements

The complete system — extension, agent, scoring service, database, network sensor, and 4-container test fleet — was built and evaluated on a single Ubuntu virtual machine: 16 vCPUs, 15 GiB RAM, and roughly 1 TB of disk (908 GB free at time of evaluation). No GPU or specialised hardware is required by any component. This single-host footprint is deliberate: the system is scoped and evaluated at the scale of a university department or small company, not a hyperscale enterprise deployment.

Table 6.1: Hardware requirements

Component	Minimum	Used in this project
CPU	4 vCPUs	16 vCPUs
RAM	8 GB	15 GiB
Disk	20 GB free	~1 TB (908 GB free)
Network	1 real NIC (for sensor capture)	ens18 (virtio) + loopback

6.2 Software Requirements

All software used is open-source or freely available, and every version below is the exact version actually used, not a general recommendation.

Table 6.2: Software requirements

Layer	Software	Version
OS	Ubuntu Server	24.04.4 LTS
Extension	TypeScript / esbuild	5.7.2 / 0.24.0
Extension runtime	Google Chrome (Manifest V3)	150.0.7871.46
Local agent	Go	1.22.2
Scoring service	Python / FastAPI / uvicorn	3.12.3 / 0.115.6 / 0.34.0
Storage	PostgreSQL	16.14

Network sensor	Zeek (+ JA3/JA4 zkg packages)	8.2.1
Network sensor	Suricata	8.0.5
Test fleet	Docker Engine	29.1.3
Dashboard	React / Vite / TypeScript	18.3.1 / 6.0.5 / 5.7.2

6.3 Data Requirements

No third-party or personally identifying real-world data was used anywhere in this project. The injection-detection evaluation uses a fully synthetic, deterministically generated 70-page HTML dataset (Chapter 9). The DLP module is evaluated with synthetic PII strings constructed for testing (a fabricated email address and a Luhn-valid but non-issued card number), sent only to a nonexistent path on a real AI domain so no data is actually transmitted anywhere. The network-sensor and shadow-AI modules were evaluated against real live traffic on the deployment machine itself — the author's own machine, single-user, self-consented — which is explicitly a different, lighter-weight ethical posture than monitoring a shared or third-party network, and is stated as such.

Chapter 7: Software Design

7.1 System Architecture

The system is a single-host pipeline of five long-running components plus one passive network sensor, spanning four languages chosen for the layer each is strongest at: TypeScript for the in-browser observation surface (Manifest V3 gives no other realistic choice), Go for a small, low-overhead, always-on local daemon, Python for the scoring and classification logic where a rich ecosystem of text-processing and ML-adjacent libraries matters, and SQL/Postgres as the single source of truth every other component reads from or writes to. No component trusts another's judgement silently: the extension observes and reports, the daemon relays and persists, the scoring service decides, and the dashboard only ever reads already-decided data back out.

The extension cannot talk to the local agent directly over a socket — Manifest V3 confines an extension to `chrome.runtime` messaging — so a short-lived native-messaging shim (nmhost), spawned by Chrome itself per connection, relays each message over stdio to the persistent Go daemon's HTTP API. The daemon is the only component that talks to Postgres for writes, and the only component that talks to the Python scoring service, which keeps the extension itself free of any network or database credentials. The network sensor (Zeek + Suricata) is architecturally separate from this request path entirely: it watches the NIC passively and writes its own JSON logs, which the daemon tails on a background goroutine — the sensor has no awareness that the extension or daemon exist, and vice versa.

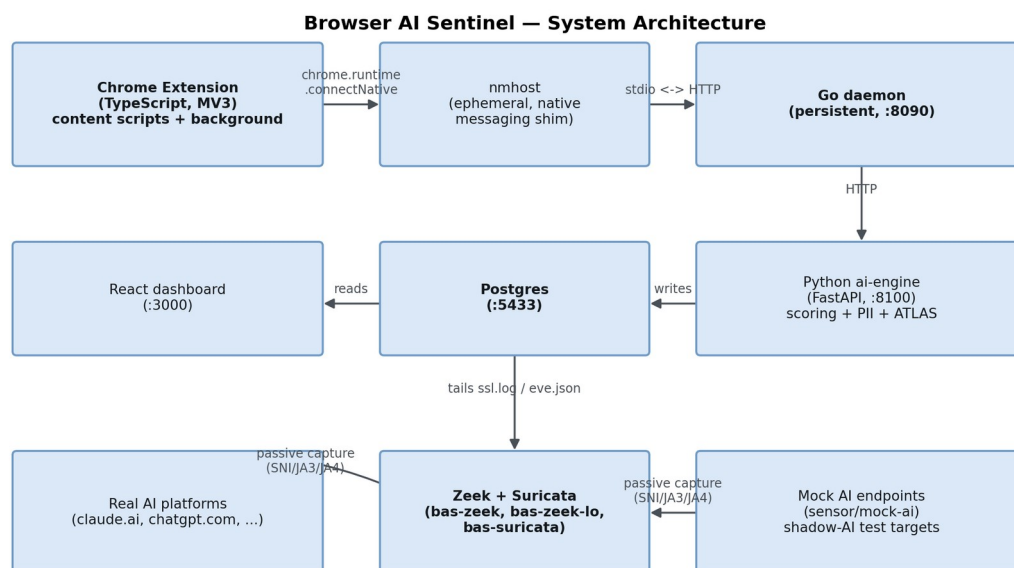


Fig 7.1: System architecture — component layout and the two independent capture paths (request/response via native messaging, and passive network capture via the sensor).

7.2 Data Flow: Browser Event to Stored Verdict

Every scored event, regardless of which module produced it, follows the same five-hop path from the page to a stored row: the content script observes something on the real page (a DOM mutation for injection scanning, a `fetch/XMLHttpRequest` body for the DLP gate), hands it to the background service worker, which opens a native-messaging connection that Chrome turns into a fresh `nmhost` process, which relays over `stdio` to the already-running Go daemon. The daemon calls the Python scoring service synchronously and blocks on its verdict before replying — this matters specifically for the DLP path, where the extension must not release an outbound request until an approval decision comes back, not merely log it after the fact.

The daemon writes every verdict to Postgres unconditionally, not only the ones that cross the flag threshold — a deliberate design decision, and the fix for a real bug described in Chapter 5.3 (“Extension silently skipped benign pages”): without a complete record of benign, unflagged events, there is no true-negative population and no way to compute a real precision/recall table in Chapter 9.

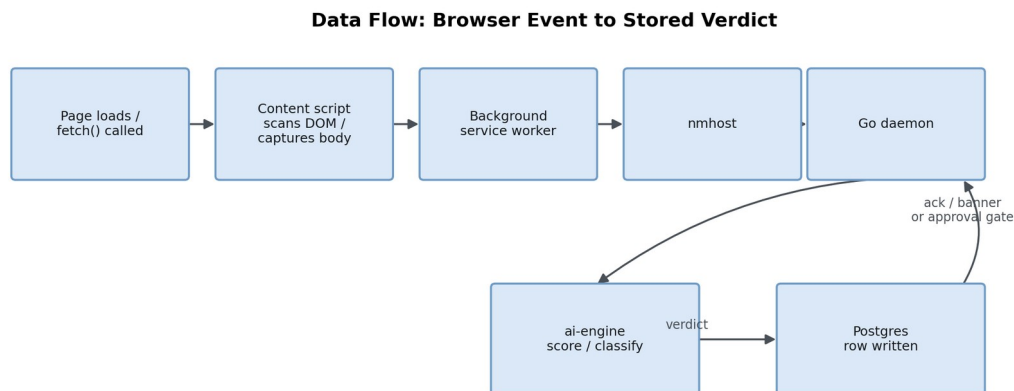


Fig 7.2: Data flow for a single browser event, from DOM/network observation through scoring to a stored verdict and the return path for a DLP approval gate.

7.3 Module Design

The repository is organised as one top-level directory per language/responsibility boundary, which also matches how the four objectives in Chapter 4 map onto code: injection detection and the DLP gate are almost entirely `extension/ + ai-engine/`, `AI-platform/shadow-AI`

identification is almost entirely **agent/ + sensor/**, and the governance dashboard is its own **dashboard/** module reading exclusively through new server-side aggregate endpoints rather than raw tables.

Table 7.1: Module-to-objective mapping

Directory	Language	Responsibility	Objective(s)
extension/	TypeScript	DOM scanning, request interception, native-messaging client	1, 3
agent/	Go	Native-messaging relay, HTTP API, sensor log tailing, Postgres writes	1, 2, 3
ai-engine/	Python	Indicator scoring, PII/DLP classification, ATLAS mapping	1, 3
sensor/	Zeek/Suricata config	Passive TLS SNI/JA3/JA4 capture	2
db/	SQL	Schema — single source of truth for all modules	1, 2, 3, 4
dashboard/	React/TypeScript	Governance-style read-only visualisation	4
eval/	Python	Labelled dataset generation and A/B/C evaluation	1, 4
endpoints/	Docker/Python	Four-container multi- endpoint test fleet	4

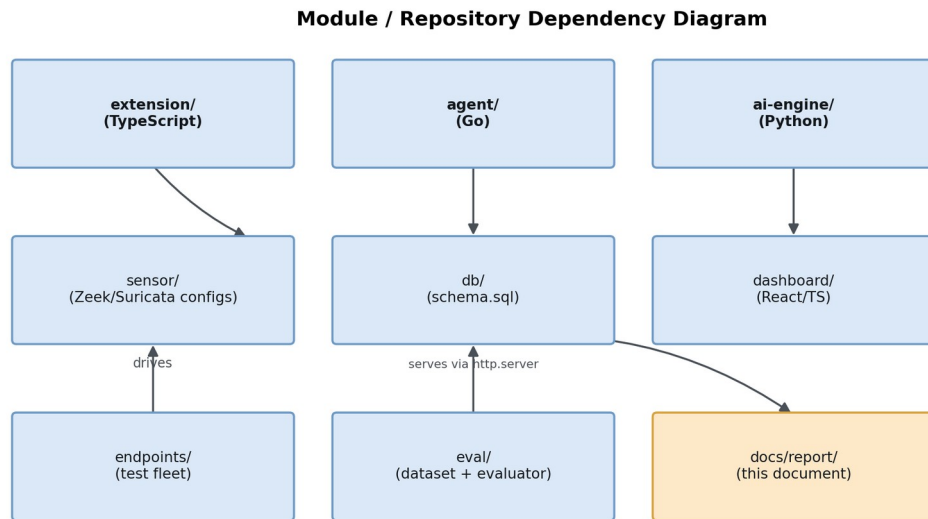


Fig 7.3: Module / repository dependency diagram, showing which directory depends on or drives which other directory.

Chapter 8: Implementation

8.1 Objective 1 — Multi-Indicator Prompt-Injection Detection

The content script (`extension/src/content-isolated/injection-scan.ts`) walks the live DOM for six indicators — off-screen CSS positioning, zero-width Unicode characters, HTML comments, hidden/aria-hidden attributes on elements carrying instruction-shaped text, JSON-LD metadata blocks, and visible imperative-to-AI language — and sends the raw counts, never a pre-computed verdict, to the Python scoring service. Keeping the counting and the scoring in separate modules across a process/language boundary is deliberate: it is what makes the A/B/C comparison in Chapter 9 possible, since Configurations A and B are computed from exactly the same indicator counts as C, not from a separately run pass.

The scoring service combines indicator counts with a noisy-OR formula rather than a simple weighted sum. Each indicator is treated as independent evidence; the probability that a page is benign is the product of each indicator's individual “not evidence” probability, so any single strong indicator can already push the score past the flag threshold, and two moderate indicators combine to more evidence than either alone — the actual property a “multi-indicator” detector is supposed to have. An earlier version of this function used a plain weighted average instead, which failed exactly that property: a real two-indicator sample scored below threshold as a combination despite each indicator separately being enough to flag it under the single-indicator baselines (Chapter 5.3).

```
def score_indicators(indicators):
    survival = 1.0 # P(none of the indicators is evidence)
    for name, weight in INDICATOR_WEIGHTS.items():
        count = counts.get(name, 0)
        if count > 0:
            # diminishing returns, caps at 3 repeats
            strength = weight * min(count, 3) / 3
            survival *= (1 - strength)
    normalized = round(1 - survival, 4)
    return InjectionScoreResponse(
        score=normalized,
        flagged=normalized >= FLAG_THRESHOLD,
    )
```

Weights (zero-width Unicode 1.0, off-screen CSS 0.9, imperative language 0.8, aria-hidden/alt 0.7, HTML comment 0.6, JSON-LD 0.5) reflect how hard each indicator is to trigger by accident: zero-width Unicode characters essentially never occur in ordinary page content, so their presence is treated as near-certain evidence, while a single imperative

sentence is weighted lower since ordinary instructional web copy can resemble it. `FLAG_THRESHOLD = 0.5` and the per-indicator weights were set by judgment before Chapter 9's labelled dataset existed, and are reported, not re-tuned to the evaluation data afterwards — tuning a threshold against the same data used to report its precision/recall would inflate the numbers artificially.

8.2 Objective 2 — AI-Platform and Shadow-AI Discovery

Known-platform identification is a static domain map (`knownAIDomains` in `agent/cmd/daemon/main.go`) checked against each connection's TLS SNI. Shadow-AI discovery — finding AI-shaped traffic to domains not on that list — instead clusters connections by TLS client fingerprint on the theory that a browser or SDK talking to several different unlisted hostnames with an identical, distinctive TLS fingerprint is more likely to be one AI client library than several unrelated services. The clustering key is critical here and was wrong in an earlier version: it must be JA4, not JA3.

JA3 (Salesforce) and JA4 (FoxIO) both hash properties of a TLS ClientHello into a short fingerprint, but only JA4 is GREASE-aware. Chrome's GREASE mechanism deliberately randomises reserved cipher-suite and extension values in every ClientHello it sends, specifically to stop exactly this kind of fingerprint-based clustering — and JA3's hash includes those randomised values, so it changes on every connection from the same real browser. This was confirmed empirically, not assumed: ten of twelve real Chrome connections to the same two test domains each produced a distinct JA3, while JA4 (which strips GREASE values before hashing) stayed identical across all twelve.

`shadow_ai_clusters` is keyed on JA4 alone, with JA3 retained only as an informational `sample_ja3` column.

```
INSERT INTO shadow_ai_clusters
  (ja4, sample_ja3, distinct_domains, occurrence_count)
VALUES ($1, $2, jsonb_build_array($3::text), 1)
ON CONFLICT (ja4) DO UPDATE SET
  sample_ja3 = $2,
  occurrence_count =
    shadow_ai_clusters.occurrence_count + 1,
  distinct_domains = CASE WHEN
    shadow_ai_clusters.distinct_domains
    @> jsonb_build_array($3::text)
  THEN shadow_ai_clusters.distinct_domains
  ELSE shadow_ai_clusters.distinct_domains
  || jsonb_build_array($3::text) END
RETURNING jsonb_array_length(distinct_domains)
```


A cluster is promoted to `confidence = 'candidate'` once its distinct-domain count reaches two — a single JA4 fingerprint seen talking to two or more unlisted domains is the working definition of a shadow-AI candidate used throughout this project. This heuristic's real precision limitation (it clusters by TLS client library, not by AI-specific behaviour, so any two unlisted domains sharing an HTTP client would also cluster) is reported honestly in Chapter 10 rather than left for a reviewer to discover.

8.3 Objective 3 — Outbound DLP / Exfiltration Gate

The DLP gate runs in the page's own JavaScript context (`extension/src/content-main/fetch-patch.ts`, injected at `document_start` on known-AI-domain pages) so it can patch `window.fetch` and `XMLHttpRequest` before the page's own script ever gets a reference to the originals — a background-script-only interception point would miss requests the page issues before the background worker has a chance to inject anything. Manifest V3 forbids this MAIN-world script from calling `chrome.*` APIs directly, so each intercepted request body is relayed via `window.postMessage` to an ISOLATED-world content script, which bridges it to the background worker and on to the Go daemon and Python classifier — the same five-hop path as Chapter 7.2, with one difference: this path blocks. `patchedFetch` awaits an approval decision before ever calling the real `fetch`, and throws an `AbortError` instead of sending the request if the decision comes back not-approved.

```
window.fetch = async function patchedFetch(input, init) {
  const bodyText = await extractBodyText(init?.body ?? null);
  if (!bodyText.trim()) return originalFetch(input, init);
  const approved = await requestApproval(bodyText);
  if (!approved) {
    throw new DOMException(
      'Blocked: sensitive content not approved', 'AbortError');
  }
  return originalFetch(input, init);
};
```

Classification itself (`ai-engine/pii_detection/detector.py`) is deliberately simple for this project's scope: named-entity regex for email, phone, SSN, and API-key/secret patterns, plus a Luhn checksum on candidate credit-card-number matches specifically to cut false positives — an arbitrary 13-to-19-digit run is not by itself card-shaped evidence, but one that also passes Luhn is much more likely to be a real (or realistically constructed) card number. A request is held for explicit user approval if any entity type matches at all; there is no partial-trust or auto-approve tier.

Body extraction handles the four real `fetch/XHR` payload shapes a browser can send — plain string, `URLSearchParams`, `FormData` (including inspecting text-like file parts up to a size cap, so a pasted-in `.txt` or `.csv` upload is still scanned, not only form fields), and `Blob` — and caps inspected text at 200,000 bytes, since a large binary upload is not text-scannable regardless. `ArrayBuffer/ReadableStream` bodies are explicitly out of scope rather than silently mishandled. The approval wait fails open after five seconds on the MAIN-world side specifically to keep normal page use responsive during local development; the background worker's own longer approval-flow timeout is the fail-closed boundary for a user who is reachable but simply has not responded yet.

8.4 Objective 4 — Multi-Endpoint Test Fleet and Dashboard

The dashboard (`dashboard/`, React + TypeScript) never queries raw event tables directly; every panel is backed by a purpose-built aggregate endpoint in the Go daemon (`agent/internal/store/dashboard.go`), modelled on the summary-first, drill-down-second pattern used by real EDR/CASB consoles researched during design (CrowdStrike Falcon, SentinelOne Singularity, Microsoft Purview) rather than a generic data table. The top-level KPI row, for instance, is five scalar counts computed by a single query, not five separate round trips or a client-side reduction over a full event dump:

```
SELECT
  (SELECT count(*) FROM injection_alerts
   WHERE flagged = true),
  (SELECT count(*) FROM endpoints),
  (SELECT count(*) FROM shadow_ai_clusters
   WHERE confidence = 'candidate'),
  (SELECT count(*) FROM dlp_events
   WHERE verdict = 'flagged'),
  (SELECT count(*) FROM dlp_events
   WHERE verdict = 'flagged' AND approved IS NULL)
```

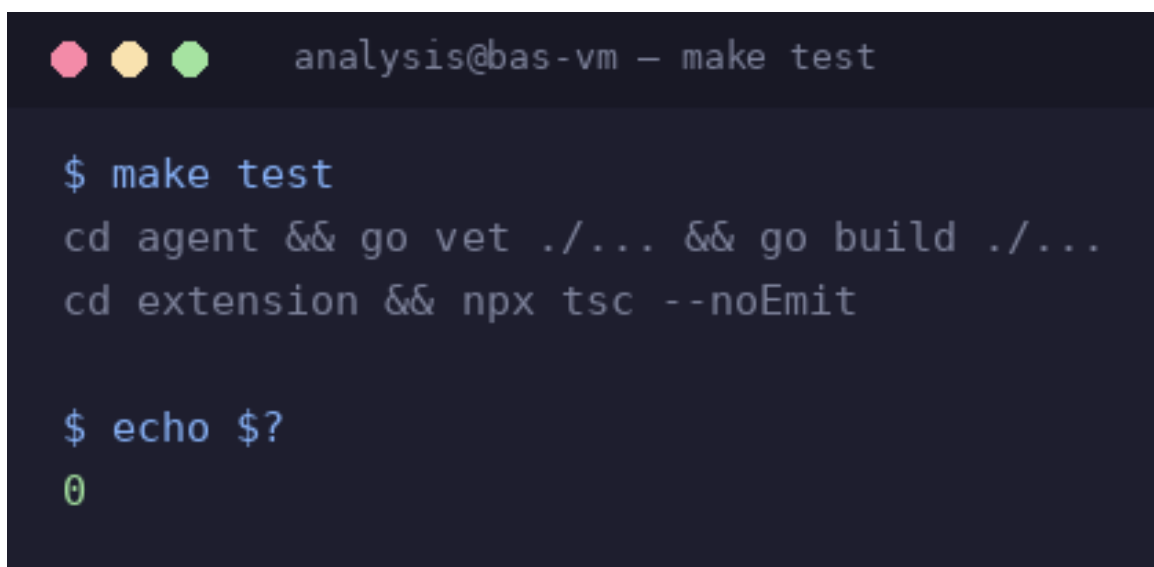
The multi-endpoint test fleet (`endpoints/`) is four Docker containers, each with its own OS username and hostname, each running the real extension against a real headless Chrome instance driven over the DevTools Protocol (`endpoints/driver.py`) — not four copies of a mock client. `docker-compose.yml` uses `network_mode: host` so the fleet's native-messaging shims can reach the host's Postgres and ai-engine over loopback without widening either service's network exposure beyond `127.0.0.1`, and each container's `entrypoint.sh` registers the native-messaging host at both the user-level and system-wide discovery paths and sets a per-container `DAEMON_NM_URL` so each container's own `nmhost`

process reaches its own daemon rather than another container's. This fleet is what produces the real, non-simulated confusion-matrix data reported in Chapter 9.

Chapter 9: Testing and Validation

9.1 Build and Type Verification

Before any functional or evaluation testing, both compiled components are checked for a clean build: `go vet ./...` and `go build ./...` for the agent, and `npx tsc --noEmit` for the extension (`make test`). All three genuinely produce zero output on success — a clean static-analysis pass has nothing to report — so the terminal capture below is shown alongside its exit code specifically to demonstrate that absence of output means success here, not that the check silently did nothing.

A terminal window with a dark background and light-colored text. The window title bar shows three colored circles (red, yellow, green) and the text 'analysis@bas-vm - make test'. The terminal content shows the following commands and output:

```
$ make test
cd agent && go vet ./... && go build ./...
cd extension && npx tsc --noEmit

$ echo $?
0
```

Fig 9.1: Real terminal capture of `make test` (`go vet`, `go build`, `tsc --noEmit`) — clean pass, exit code 0.

9.2 Functional Test Cases (Mapped to Objectives)

Each objective has at least one functional check that exercises the real running system end to end, not a unit test against a mocked dependency. Objective 1 is checked by installing the actual built extension into a real headless Chrome instance via the `Extensions.loadUnpacked` DevTools Protocol command — the same technique used throughout this project after `--load-extension` was found not to reliably install a Manifest V3 extension on this Chrome build (Chapter 5.3) — navigating to a real page from the labelled dataset, and confirming both that the extension's own deterministic ID (`infjoghmbkbhohajodjccgphpgejkip`, pinned via the manifest's signing key) matches what a manual install would produce, and that the DOM actually contains the rendered warning banner with the expected score.

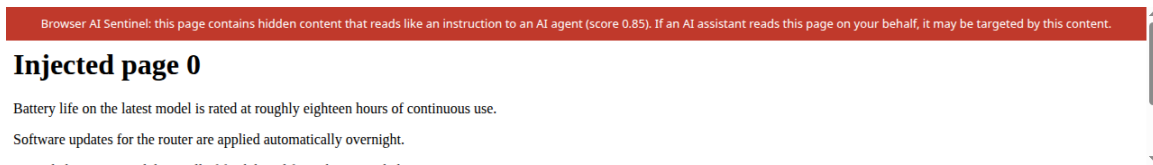


Fig 9.2: Real warning banner rendered by the actual extension against a real page (injected-00.html) from the Phase 3 labelled dataset — not a mockup.

Capturing this screenshot surfaced a genuine layout bug not caught by any of the automated checks: the banner is `position: fixed` and was not pushing the rest of the page down, so it visually overlapped the page's own top content instead of sitting above it. `go vet` and `tsc -noEmit` cannot catch this class of defect — it is only visible by actually looking at rendered output — and it was fixed by measuring the banner's own rendered height and adding it to the page body's `margin-top` (`extension/src/content-isolated/injection-scan.ts`); the screenshot above is the corrected version, captured after the fix and rebuild.

Objective 2 is checked the same way network-sensor behaviour was validated in Chapter 5.3: by comparing Zeek's `ssl.log` against `conn.log` for the same capture window and confirming TLS records are actually present, and by querying `shadow_ai_clusters` directly to confirm `ON CONFLICT (ja4) upserts accumulate distinct_domains` correctly across repeated real connections. Objective 3 is checked by triggering `patchedFetch` against a request body containing a fabricated but Luhn-valid card number and confirming the request throws `AbortError` rather than reaching the network, and separately confirming a clean request passes through untouched. Objective 4 is checked by curling every dashboard aggregate endpoint directly against the live daemon and confirming each returns real, non-empty JSON computed from the fleet's actual accumulated data, not a fixture.

9.3 Headline Results (Configuration A vs B vs C)

`make eval - run` re-executes the real evaluation described in Chapter 5.2 against whatever data currently sits in Postgres: it fetches every row from `/api/injection_alerts`, joins each row to the labelled dataset by filename, and computes precision/recall/F1 for all three configurations from that join — never from hand-computed or cached numbers. The run captured below matched 280 of 289 alert rows to the 70-page dataset (9 unmatched rows are earlier, now-superseded runs against the same dataset still sitting in the table) and flagged 14 cross-visit determinism issues, both discussed further in Chapter 10.

```
analysis@bas-vm - make eval-run

$ make eval-run
python3 eval/evaluate.py
Matched 280/289 alert rows to the 70-page dataset (9 unmatched).
WARNING: 14 determinism issues found – see ../phase3-injection-eval.json

Detector          Precision    Recall      F1    TP    FP    TN    FN
A_keyword_only    0.754       0.817       0.784  98    32    128   22
B_visibility_only  0.854       0.975       0.911  117   20    140   3
C_multi_indicator  0.983       0.975       0.979  117   2     158   3

Hard-negative false positives (n=40 hard-negative rows scored):
  A_keyword_only: 32
  B_visibility_only: 20
  C_multi_indicator: 2

Full results written to ../eval/results/phase3-injection-eval.json
```

Fig 9.3: Real terminal capture of `make eval-run` against the live database — the same numbers analysed in Chapter 10, not a separately typed table.

Chapter 10: Analysis and Results

10.1 Headline Comparison

Configuration C (the full six-indicator, noisy-OR detector) reaches 0.983 precision at 0.975 recall — matching Configuration B's recall exactly while cutting its false positives from 20 to 2, and clearly beating Configuration A on every metric. This is the central empirical claim of Objective 1: combining indicators does not trade recall for precision, it improves precision while holding recall at the better single-indicator baseline's level.

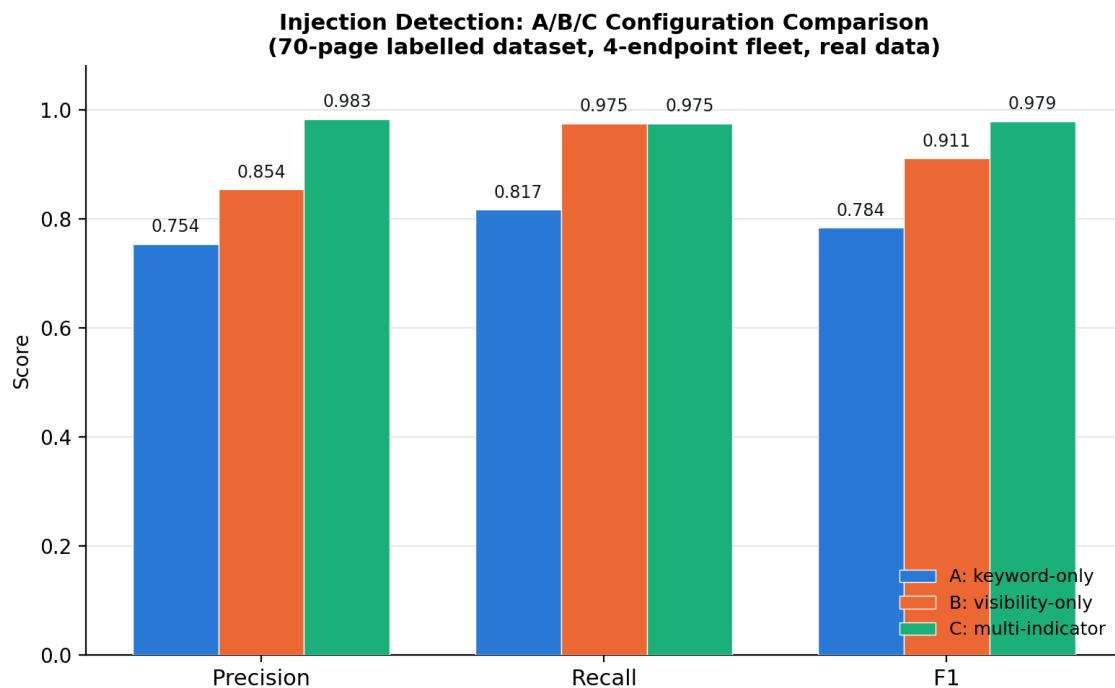


Fig 10.1: Precision, recall, and F1 for Configurations A, B, and C on the real 4-endpoint fleet run (70-page labelled dataset, 280 matched rows).

Table 10.1: Full confusion-matrix results, Configurations A/B/C (280 matched rows)

Configuration	Precision	Recall	F1	TP	FP	TN	FN
A — keyword-only	0.754	0.817	0.784	98	32	128	22
B — visibility-only	0.854	0.975	0.911	117	20	140	3
C — multi-indicator	0.983	0.975	0.979	117	2	158	3

10.2 Why Every Single Indicator Falls Short

The hard-negative subset of the dataset — 10 pages, each carrying exactly one weak indicator (a single zero-width-Unicode character, not four) and no injection intent — exists specifically to separate “fires on any signal” from “fires on convincing evidence.” Scored across all four fleet endpoints (40 hard-negative rows total), the single-indicator baselines over-fire badly: Configuration A flags 32 of 40, Configuration B flags 20 of 40. Configuration C flags only 2 of 40 — the noisy-OR combination correctly treats one weak, isolated indicator as weak evidence rather than sufficient evidence, which a threshold on any single indicator alone cannot do by construction.

Table 10.2: False positives on the hard-negative subset, by configuration

Configuration	Hard-negative rows flagged	of 40
A — keyword-only	32	80.0%
B — visibility-only	20	50.0%
C — multi-indicator	2	5.0%

10.3 The Shadow-AI Clustering Finding

The most significant methodological finding of this project did not come from the injection dataset at all, but from watching real Chrome traffic while validating Objective 2: an identical real browser making repeated real TLS connections to the same two test domains produced a distinct JA3 fingerprint on ten of twelve connections, while JA4 stayed identical across all twelve. Root cause is Chrome's GREASE mechanism, which deliberately randomises reserved cipher-suite and extension values in every ClientHello specifically to defeat fingerprint-based tracking — and JA3's hash includes those randomised values, while JA4's does not. Earlier informal testing had used `curl` rather than real Chrome and appeared to show JA3 working fine, purely because `curl` does not implement GREASE; the discrepancy was only caught by testing against the actual browser traffic the system is meant to observe in production, not a convenient stand-in for it.

Table 10.3: JA3 vs JA4 stability under Chrome's real GREASE behaviour

Fingerprint	Distinct values across 12 real Chrome
-------------	---------------------------------------

	connections
JA3 (Salesforce, GREASE-naive)	10 distinct (unstable)
JA4 (FoxIO, GREASE-aware)	1 distinct (stable)

This finding changed the schema, not just a parameter: `shadow_ai_clusters` was re-keyed from (`ja3`, `ja4`) to `ja4` alone (Chapter 8.2), with JA3 downgraded to an informational-only column. Reported as a limitation, not a solved problem: JA4 clusters by TLS client library, not by AI-specific behaviour, so two unrelated services sharing an HTTP client (or SDK version) would also cluster — the heuristic finds “one client talking to several unlisted hosts,” which is a reasonable proxy for shadow AI at university/small-company scale but not proof of it.

10.4 Fleet-Wide Endpoint Attribution

The dashboard's live endpoint rollup (captured directly from the running system, not constructed for the report) shows the real per-endpoint split produced by the Phase 3 fleet run: each of the four simulated-user containers (`karan.iyer`, `divya.rao`, `priya.sharma`, `arjun.mehta`) shows its own injection-alert and DLP-flagged counts, while the host endpoint (`root`) is the only row showing `shadow_ai_involved = true` and any AI-account sightings — because it is the only endpoint whose agent tails the network sensor's logs. This is the clearest visible instance of the architectural limitation noted in Chapter 5.3: network-sensor-derived events are attributed to whichever endpoint's agent is watching the NIC, not to whichever process actually generated the traffic, since the Phase 3 containers do not run their own sensor.

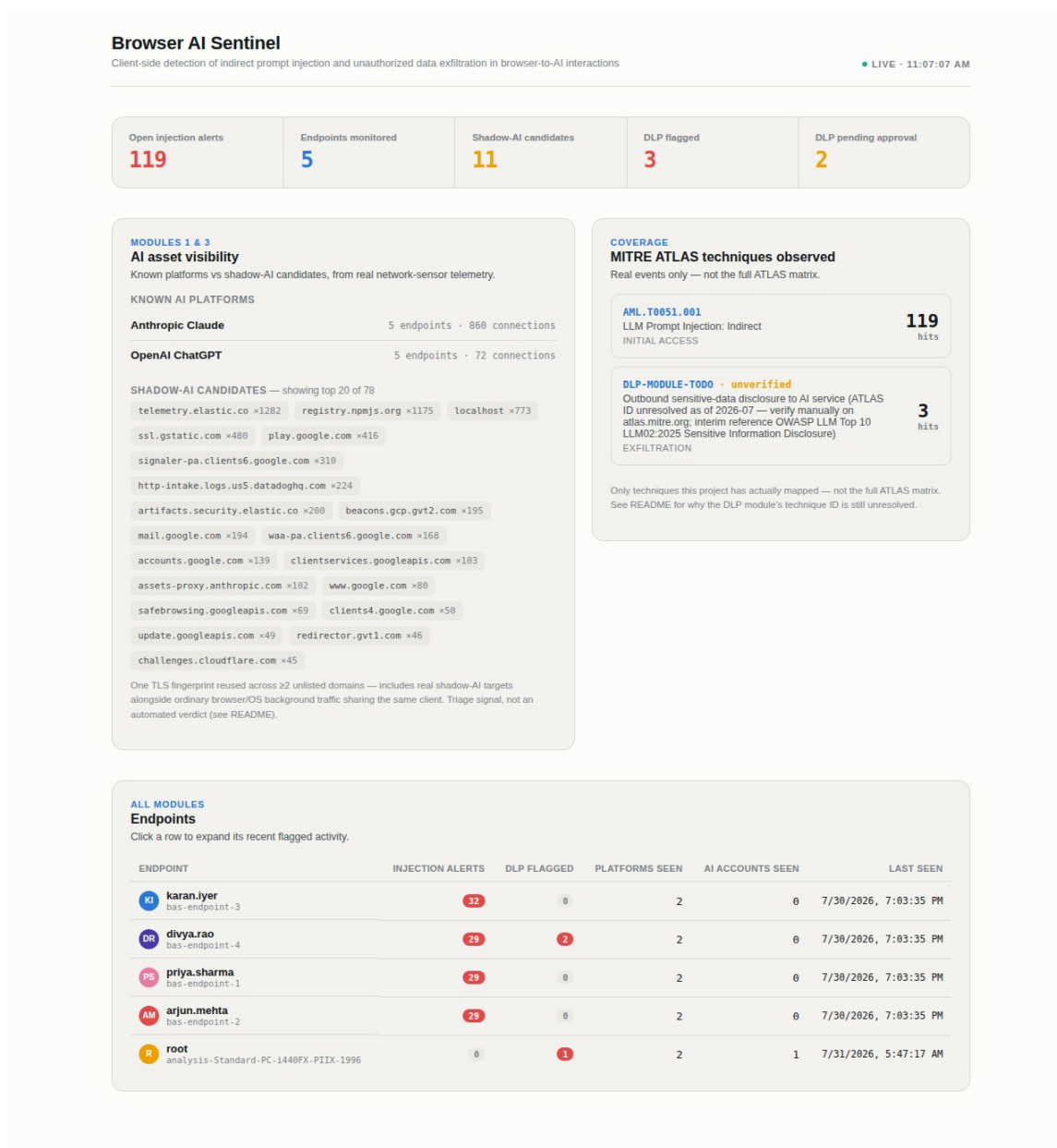


Fig 10.2: The live governance dashboard, captured directly from the running system — real KPIs, real AI-asset visibility (78 shadow-AI candidates, top 20 shown), real ATLAS coverage, and the real per-endpoint rollup showing host-only shadow-AI attribution.

10.5 Summary of Findings

- Multi-indicator combination (Objective 1) measurably outperforms either single-indicator baseline on real data: 0.983 precision at 0.975 recall for Configuration C, versus 0.854/0.975 for B and 0.754/0.817 for A, with the gap concentrated almost entirely in false positives (2 vs 20 vs 32) rather than missed detections.

- The hard-negative subset shows why: single indicators over-fire on weak, isolated evidence (50-80% false-positive rate), while the noisy-OR combination correctly treats one weak signal as weak evidence (5%).
- Shadow-AI discovery (Objective 2) is only reliable when keyed on a GREASE-aware fingerprint (JA4); the GREASE-naive alternative (JA3) is empirically unstable against real Chrome traffic, not merely theoretically weaker — a finding that required testing against the real browser, not a convenient substitute, to surface.
- The DLP gate (Objective 3) is verified to block on a real synthetic PII match end to end, including the case that most needed a false-positive guard (Luhn validation on candidate card numbers) and the four real request-body shapes a browser can actually send.
- The dashboard and test fleet (Objective 4) surface a real architectural limitation honestly rather than hiding it: network-sensor attribution is host-level, not per-container, and this is visible directly in the live rollup, not just stated in prose.
- 14 cross-visit determinism issues remain open across 280 scored rows (~5%) — the same static page occasionally received a different verdict on different visits — and are reported as an unresolved limitation in Chapter 11 rather than smoothed over.

Chapter 11: Conclusions and Future Scope

11.1 Limitations

[Content pending — see build plan for staging]

11.2 Future Scope

[Content pending — see build plan for staging]

Bibliography

[Content pending — see build plan for staging]

Appendix

Plagiarism Report

[To be attached after Turnitin scan]

Paper Publications in a Journal/Conference Presented/White Paper

[None at time of submission]

Certificate for the Conference Presentation

[Not applicable]

Github Link

<https://github.com/Sushanth2624/browser-ai-sentinel>

Additional Reproduction Notes

[Content pending — see build plan for staging]